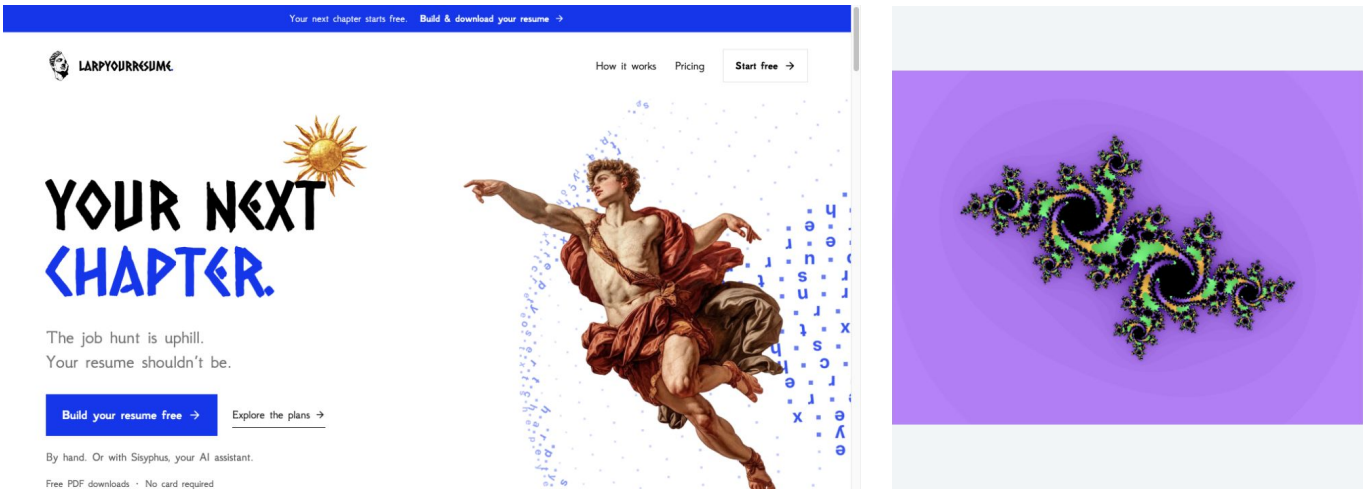
ENGINEERING PORTFOLIO

Arda Tas

Computer Science at the Technical University of Munich | Munich, Germany

Selected work in software engineering

I build applications that connect clear interfaces with reliable execution. My work spans AI-assisted products, industrial data processing, and performance-focused C programming.



Product interface: larpyourresume. Fractal output: Julia Set Explorer.

A GUIDE TO THE WORK

02-03 **larpyourresume**
Full-stack product, PDF generation, visual design

04 **Werkio**
Reviewed AI workflows and browser automation

05 **CHECK24 MCP App**
API contracts, tool routing, conformance feedback

06 **Industrial data pipeline**
Scientific Python, GUI workflows, data integrity

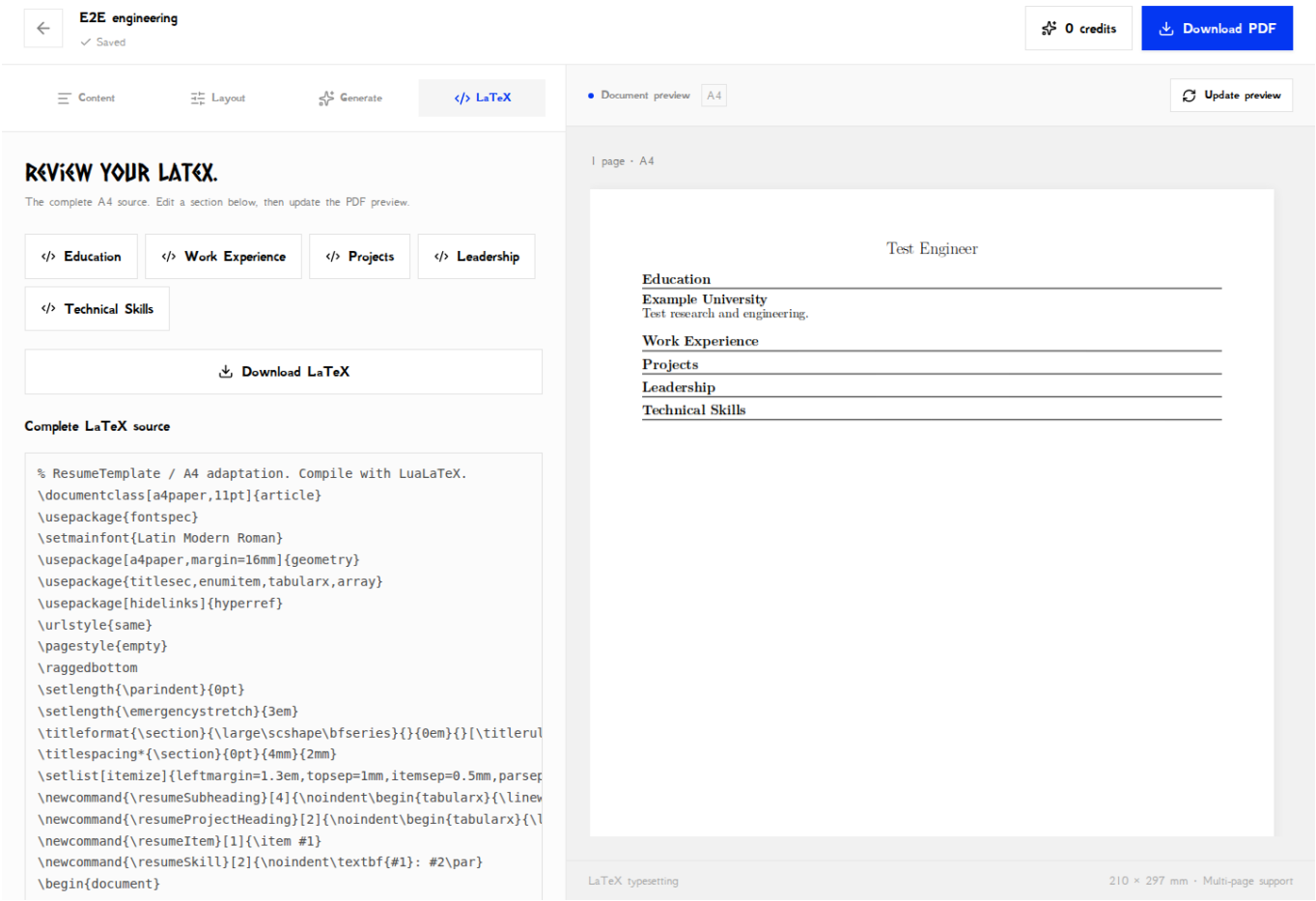
07 **Julia Set Explorer**
SIMD profiling, correctness, measured performance

Portfolio GitHub LinkedIn arda@ardatas.com

01 / PRODUCT ENGINEERING

larpyourresume

A resume workspace with structured editing, LaTeX controls, and compiled A4 PDF output.



Application screenshot with test content: the LaTeX editor and compiled PDF preview.

WHAT I BUILT

I built the resume dashboard, autosaving editor, and PDF workflow. Users can organize multiple versions, edit section forms or LaTeX, and control page margins, typography, and spacing.

ENGINEERING DECISIONS

A separate container compiles LuaLaTeX with shell access disabled and bounded requests. The UI distinguishes saved edits from the last compiled preview. Save recovery preserves conflicting edits as a separate resume.

Stack: Next.js, React, TypeScript, Zod, Supabase/PostgreSQL, Docker, LuaLaTeX.

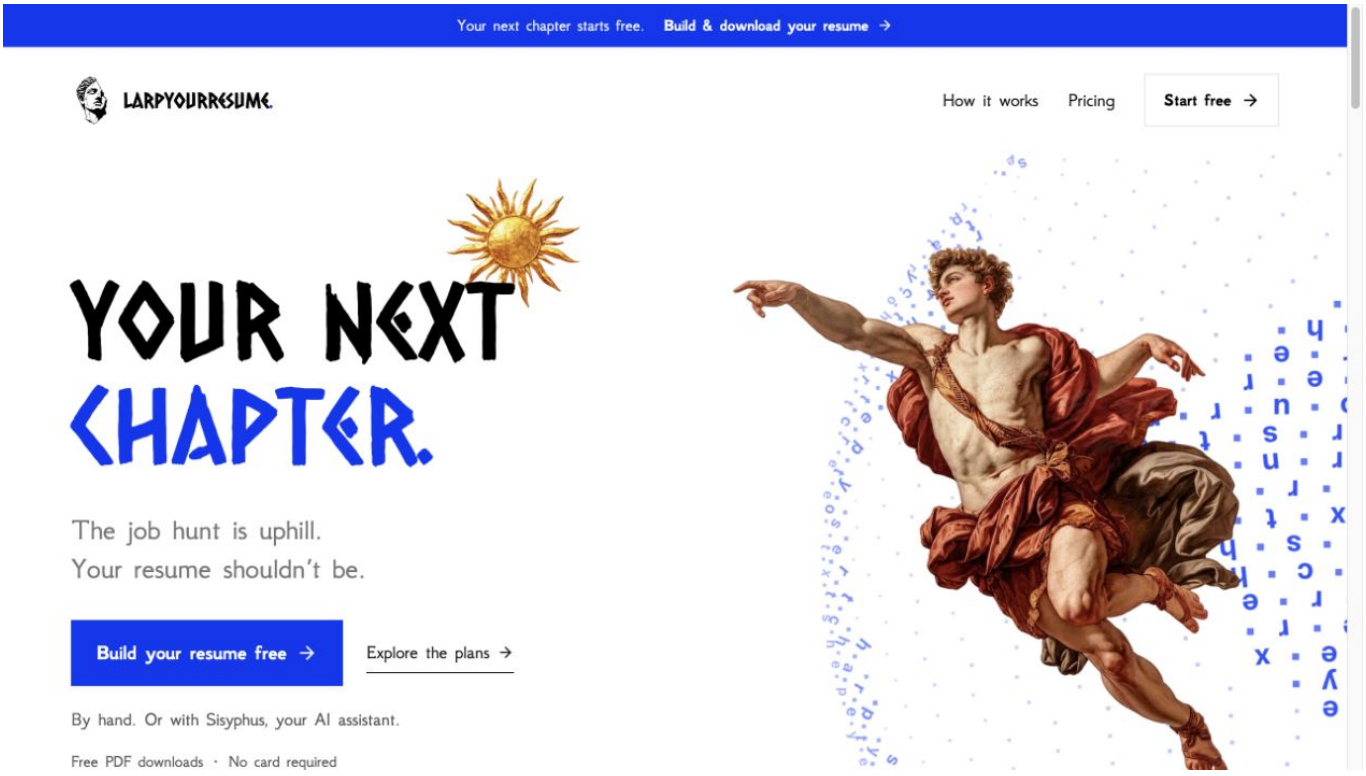
LOOK CLOSER The editor displays the compiled document. Downloading compiles the latest content.

Open product Original LaTeX template

01 / INTERFACE AND AI WORKFLOW

Designing the resume workspace

A visual identity and editing flow built around a concrete task: preparing the next application.



Live landing page, captured 13 October 2026. The interface uses a black, white, and blue palette.

VISUAL DESIGN

I developed the product interface and landing page around classical imagery and a restrained color palette. Large display type establishes the identity; simple labels and structured controls explain the workflow.

SISYPHUS AI ASSISTANT

The implemented generation flow combines resume text with job context and user instructions. Typed document contracts constrain the output before it enters the editor. Users can continue editing the generated content.

AVAILABILITY

The free editor and PDF downloads are publicly offered. The live site currently labels AI allowances and paid plans as coming soon.

LOOK CLOSER The same product connects visual design, document editing, and the underlying generation workflow.

Explore the workspace and design

02 / FULL-STACK AI WORKFLOWS

Werkio

A student job platform and browser extension for reviewed, AI-assisted applications.

APPLICATION WORKFLOW / SCHEMATIC

1. Read the application form

The extension extracts supported visible fields and job context.

↓

2. Prepare a review plan

The backend validates proposed values and their supporting evidence.

↓

3. Review and fill

The user checks the plan; the extension applies supported values.

↓

4. Submit manually

Final submission remains a user action.

MY CONTRIBUTION

I built the React frontend, Spring Boot backend, and TypeScript extension. Gemini supports form mapping and cover-letter drafting from approved facts. Workflow state explains failures and invalidates stale plans when the page changes.

RELIABILITY WORK

I added structured-output validation, evidence checks, access controls, and CSRF protection. Playwright scenarios cover preserving user edits, sensitive-field handling, provider failures, and manual submission. These are controlled test fixtures.

Stack: React, Java/Spring Boot, TypeScript, PostgreSQL, Flyway, Gemini, Playwright.

LOOK CLOSER

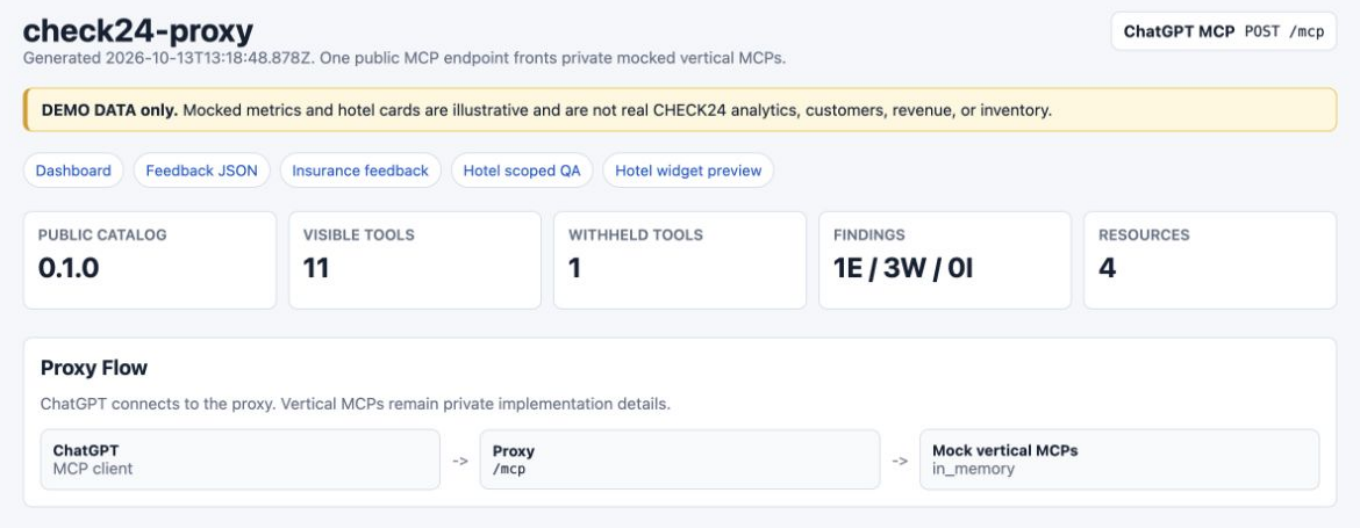
The difficult part is the boundary between an AI suggestion and a permitted browser action.

Open Werkio Project description

03 / PROTOCOL AND API ENGINEERING

CHECK24 MCP App

One ChatGPT-facing Model Context Protocol endpoint for four mocked product services.



Local application screenshot, 13 October 2026. The demo catalog exposes 11 tools and withholds one invalid tool. All product data is mocked.

WHAT I BUILT

For the CHECK24 GenDev scholarship task, I built a TypeScript proxy for hotel, flight, package-holiday, and insurance tools. Stable namespaced tool names map to the original service tools behind one public catalog.

VALIDATION AT THE BOUNDARY

AJV validates tool schemas and incoming arguments. If a tool declares structured output, the proxy validates that too. Invalid tools are withheld with a rule, severity, and a suggested fix exposed in JSON and the dashboard.

A CONCRETE FAILURE CASE

The insurance mock intentionally omits an input schema for one tool. The proxy withholds it and reports `inputSchema_missing`. Reviewers can inspect the same validation boundary through service-specific QA routes.

Stack: TypeScript, Node.js, Express, MCP SDK, AJV, Vitest, Docker Compose.

LOOK CLOSER A usable tool catalog needs explicit validation failures and traceable routing, as well as aggregation.

Public source and setup instructions

04 / RESEARCH SOFTWARE

Industrial data pipeline

Student assistant work at TUM AIS: visual workflows for industrial time-series processing.

SYSTEM ARCHITECTURE / SCHEMATIC



HDF5 output structure

```
Raw-Data/.../MeasurementData

Runs/<run-id>/.../MeasurementData
```

Each compound record keeps Timestamp_float and Value together. Timestamped run groups retain outputs from successive executions.

Conceptual diagram based on the implementation. No industrial measurements are shown.

MY CONTRIBUTION

I develop the desktop workflow application and integrate it with the existing Python processing pipeline. My work includes connected workflow chains, GUI-to-JSON compilation, HDF5 layout updates, and documentation of the shared contracts.

DATA INTEGRITY IN PRACTICE

CSV import sorts whole compound records by timestamp, so time/value pairs stay aligned. Processing runs can use a separate HDF5 working copy. Raw inputs remain available while timestamped result groups support comparison between runs.

Stack: Python, PySide6, NumPy, SciPy, h5py/HDF5.
Scope: contribution to an existing research codebase; the diagram summarizes the integration.

LOOK CLOSER

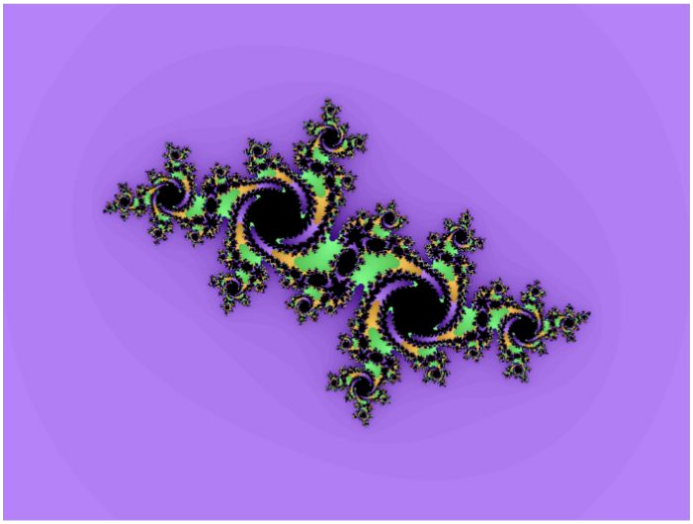
The visual workflow, executable JSON, and stored data must agree on addressing and structure.

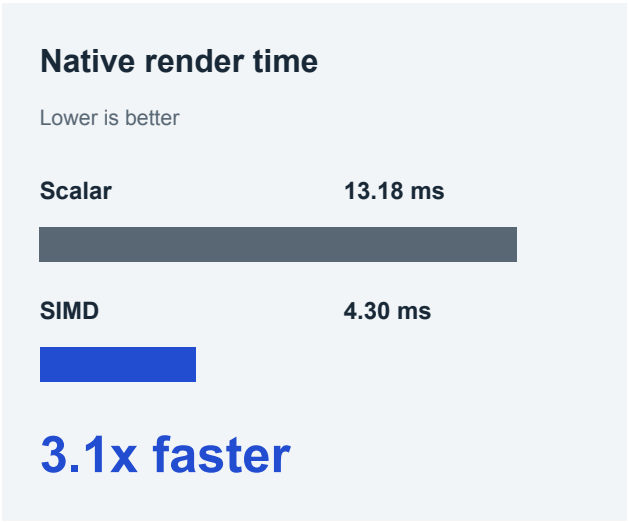
Project description TUM AIS

05 / PERFORMANCE ENGINEERING

Julia Set Explorer

A team-built C renderer, with an interactive WebAssembly version on my portfolio.





Left: color output captured from the browser explorer. Right: the team report's native C benchmark, using grayscale output; these are separate rendering contexts.

MY CONTRIBUTION

I profiled the SIMD implementation and investigated the active-mask bottleneck. I fixed escaped-lane reactivation, replaced masked addition with subtraction, added monotonic timing and BMP dimension checks, and designed correctness and performance experiments.

HOW THE RENDERER WORKS

SIMD processes four pixels in parallel. Each lane iterates the Julia recurrence until it escapes or reaches the iteration limit. Active masks keep escaped lanes inactive. A scalar fallback handles widths that do not fill a complete SIMD group.

BENCHMARK AND CORRECTNESS EVIDENCE

Reported mean over 100 runs: **13.18 ms scalar** and **4.30 ms SIMD** at 800 x 600 pixels and 100 iterations. Hardware: AMD Ryzen 9 5950X; compiler: GCC 14.2.0, C17, -O3, -msse4.2. The team report records byte-identical SHA-256 comparisons across more than 30 scenarios.

The core SIMD implementation was written by a teammate. My focus was profiling, correctness fixes, experiments, and performance visualization. Timings are reported project results.

LOOK CLOSER Performance work needs a reference implementation and tests for lane divergence and edge cases.

Interactive explorer **Public source, report, and contributions**